

Activation function comparison in neural-symbolic integration

Cite as: AIP Conference Proceedings **1750**, 020013 (2016); <https://doi.org/10.1063/1.4954526>
Published Online: 21 June 2016

Mohd. Asyraf Mansor, and Saratha Sathasivam



View Online



Export Citation

ARTICLES YOU MAY BE INTERESTED IN

[Performance analysis of activation function in higher order logic programming](#)

AIP Conference Proceedings **1750**, 030007 (2016); <https://doi.org/10.1063/1.4954543>

[Accelerating activation function in higher order logic programming](#)

AIP Conference Proceedings **1750**, 030008 (2016); <https://doi.org/10.1063/1.4954544>

[Satisfiability based reverse analysis method in diabetes detection](#)

AIP Conference Proceedings **1974**, 020020 (2018); <https://doi.org/10.1063/1.5041551>

Lock-in Amplifiers
up to 600 MHz



Activation Function Comparison in Neural-Symbolic Integration

Mohd. Asyraf Mansor^{1, a)} and Saratha Sathasivam^{1, b)}

¹*School of Mathematical Sciences, Universiti Sains Malaysia,
11800 Minden, Pulau Pinang, Malaysia.*

^{a)}Corresponding author: asyrafalvez@live.com

^{b)}saratha@usm.my

Abstract. The activation function is a dynamic paradigm for doing logic programming in Hopfield neural network. In neural-symbolic integration, the activation function used to metamorphose the activation level of a unit (neuron) into an output signal. The proposed activation function is Bipolar sigmoid activation function. The main goal of this research is to compare and analyze the performance of proposed activation function paradigm with another sign function, namely McCulloch-Pitts function based on Wan Abdullah's method. In this study, we evaluate experimentally the differences between both functions through computer simulations. Computer simulations are conducted to demonstrate the ability of Bipolar sigmoid function and McCulloch-Pitts function doing the logic programming in Hopfield neural network. Microsoft Visual C++ 2013 is used as a platform for training and testing. The performance of Bipolar sigmoid activation function and the McCulloch-Pitts function were discussed holistically by comparing the global minima ratio, Hamming distance and training or computation time. It was proven by computer simulations that the Bipolar sigmoid activation function has a better performance, provides good solutions and achieves an acceptable stability compared to the McCulloch-Pitts function.

INTRODUCTION

Artificial intelligence is one of the most celebrated fields in mathematical computation and computer science community [1]. Strictly speaking, neural-symbolic integration is an essential model in artificial intelligence. In neural-symbolic field, the benefits of both neural network and logic programming can be combined [2, 3]. In this paper, we emphasized on three thought-provoking aspects in artificial intelligence namely, Hopfield network, activation function and logic programming.

There are countless types of neural networks, from the simple to intricate, just like the evolution human biological nervous system [4]. A Hopfield network [5] is a recurrent neural network invented by John Hopfield. It comprised of a set of N interconnected neurons which all neurons are connected to all others in both directions [1-3]. Then, the Hopfield neural network is a simple recurrent network which can work as an efficient associative memory, and it can store certain memories in a manner rather analogous to the brain [6]. Additionally, it is a branch of neural network that assist the researchers in constructing the model of our brain.

Basically, logic programming can be treated as a problem in combinatorial optimization standpoint [3, 7]. Therefore, it can be carried out in a neural network to obtain desired solutions. In this paper, we will consider logic programming in Hopfield network. The conventional paradigm is Wan Abdullah's logic programming based on McCulloch-Pitts function. Wan Abdullah [8] proposed a paradigm of doing logic program on a Hopfield network. We extended the work related on the neural-symbolic integration by implementing Hopfield network and activation function in doing logic program [9]. As a result, we proposed the Bipolar sigmoid activation function as the catalyst of doing logic programming in Hopfield network. The Bipolar sigmoid activation function will be compared with the McCulloch-Pitts function by using computer simulations. The robust model will be used to solve various combinatorial optimization problems more effectively.

This paper is organized as follows: Section 2, introduces the fundamental concept of Hopfield model. In Section 3, the procedure of doing logic programming is discussed. Besides, Section 4 covers brief discussion of activation functions. In section 5, theory implementation of the activation functions are been discussed. Finally, Section 6 and 7 enclose the experimental results and conclusion of this research.

HOPFIELD NEURAL NETWORKS

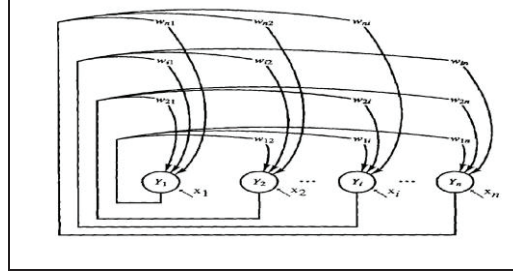


FIGURE 1. Illustration of discrete Hopfield network

As an expended form of Hopfield network representation, discrete Hopfield network is shown as in Figure 1. Hopfield [5] highlighted that discrete Hopfield network is compulsory in solving various combinatorial optimization problems as a content addressable memory. The Hopfield models consist of N formal neurons that represented as variable $x_i(t), i = 1, 2, \dots, N$. Neurons are bipolar, so the activation value for variable had declared as $x_i \in \{-1, 1\}$, comply with the dynamics $x_i \rightarrow \text{sgn}(h_i)$ where sgn is signum function [9]. The induced local field is $h_i = \sum_j w_{ij} x_j + w_i$ where w_i a fixed bias (negative of the threshold) is applied externally to neuron j . w_{ij} is the synaptic strength from neuron i to j . The connections had controlled to be symmetric and zero-diagonal $w_{ij} = w_{ji}, w_{ii} = 0$

The energy function for the discrete Hopfield networks for second order Hopfield networks is given

$$E = -\frac{1}{2} \sum_i \sum_j w_{ij} x_i x_j - \sum_i w_i x_i \quad (1)$$

which the Lyapunov function is decrease monotonically with the dynamics. The two-connection model can be generalized to include higher order connections. This transforms the “field” into

$$h_i = \left(\dots + \sum_j \sum_k w_{ijk} x_j x_k + \sum_j w_{ij} x_j + w_i \right)_i \quad (2)$$

where “.....” denotes still higher orders, and an energy function can be written as follows:

$$E = \dots - \frac{1}{3} \sum_i \sum_j \sum_k w_{ijk} x_i x_j x_k - \frac{1}{2} \sum_i \sum_j w_{ij} x_i x_j - \sum_i w_i x_i \quad (3)$$

Provided that $J_{ijk} = J[ijk]$ for i, j, k distinct, with [...] denoting permutations in cyclic order, and $J_{ijk} = 0$ for any i, j, k equal, and that similar symmetry requirements are satisfied for higher order connections. The updating rule maintains

$$x_i(t+1) = \text{sgn}[h_i(t)] \quad (4)$$

LOGIC PROGRAMMING

Logic programming is one of the most important elements in Neural-Symbolic integration. In layman's term, logic is deals with true and false while in the logic programming, a set of Horn clauses that formed by atoms are represented to find the truth values of the atoms in the clauses [10].

Meanwhile, Wan Abdullah's method [4, 8] quests for the best solutions, given the clauses in the logic program, and the corresponding solutions may change as new clauses added. Even when clauses in the logic program are inconsistent, but yet by using Wan Abdullah's method, interpretation with least logical inconsistent can be obtained. However, the main different between these both approaches are: Pinkas's [7] work revolves around first-order logic meanwhile Wan Abdullah's work revolves around propositional Horn clauses and learning ability of the Hopfield network. In this paper, we are more concerned with program clauses and learning ability, by addressing Wan Abdullah's method.

We use neurons to store the truth value of atoms to write a cost function for minimization when all the clauses are satisfied. In our research, we limit the investigation until NC3 (number of clauses = 3). Based on Wan Abdullah's method [8], the following algorithms review on logic programming can be implemented in the Hopfield network.

- i) Given a logic program, translate all the clauses in the logic program into basic Boolean algebraic form. For example, $A \leftarrow B, C$ as $A \vee \neg (B \wedge C) = A \vee \neg B \vee \neg C$
- ii) Identify a neuron to each ground neuron.
- iii) Initialize all connections strengths to zero. It assumed the connection with A, B and C is zero value.
- iv) Derive a cost function that is associated with the negation of all clauses, such that $\frac{1}{2}(1 + S_i)$ represents the logical value of a neuron X, where S_i is the neuron corresponding to X. S_i carries the values of 1 if X is true. In contrary, it will carry value of -1 if X is false. Negation (neuron X does not occur) is represented by $\frac{1}{2}(1 - S_i)$; multiplication represents the conjunction logical connective whereas addition represents disjunction logical connective.
- v) Comparing the cost function with energy by obtaining the values of connecting strengths.
- vi) Let the neural network to progress until minimum energy is reached. The neural states then provide a solution interpretation for the logic program. Next, verify whether the solution obtained is a global solution or not.

ACTIVATION FUNCTION

The McCulloch-Pitts function is a conventional paradigm of doing logic programming in Hopfield network proposed by Wan Abdullah [8]. According to [11], the solutions of McCulloch-Pitts function usually stuck in local minima of the energy configuration and it is time consuming to obtain global solutions. Hence, it doesn't decrease the computation burden for training the network [12]. As a result, we introduced Bipolar sigmoid activation function as an alternative to overcome the previous drawbacks. The Bipolar activation function used to convert the activation level of a unit (neuron) into an output signal. It is also known as transfer function or squashing function due to the capability to squeeze the amplitude range of output signal to some finite value [13].

Bipolar Sigmoid Activation Function

Bipolar sigmoid is a type of activation function or "squashing function" that is widely used in Hopfield neural network. This function is related to the sigmoid function. For this type of activation function, it goes well for applications that produce output values in the range of $[-1, 1]$ as it is a bounded function. In addition, each neuron's activation is between -1 to 1. This function can be a good choice to train the data between 0 to 1 compared with the McCulloch-Pitts function. As a result, Bipolar sigmoid activation function is easier to differentiate and monotonic [14].

The graphical representation of this activation function is as follows:

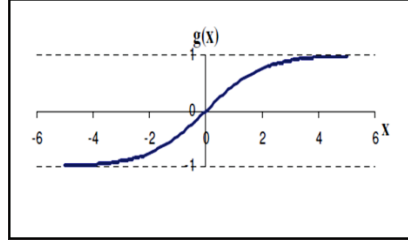


FIGURE 2. Graphical representation of Bipolar sigmoid activation function

In order to compute the output, we use the value of local field by applying equation (2) into equation (5).

$$y(x) = g(h_i) \quad (5)$$

Bipolar sigmoid activation function is given by

$$g(h_i) = \frac{1 - e^{-h_i}}{1 + e^{-h_i}} \quad (6)$$

Let $y(x)$ represents as actual output

$$y(x) = \begin{cases} -1, & g(h_i) < 0 \\ +1, & g(h_i) \geq 0 \end{cases} \quad (7)$$

In this case, we punch in the local field value into Bipolar Sigmoid activation function. If it exceeds the threshold ($\theta = 0$) the actual output is 1 else the value of the actual output is -1. After the network computes the output pattern, and if there is an error, or in other words a difference between actual and desired output patterns, the weights are adjusted to reduce this error [9, 15].

THEORY IMPLEMENTATION

The implementation involves a precise procedure. Firstly, the random program clauses are generated. Secondly, the initial states are being initialized for the neurons in the clauses. Then, initialize the connection strengths to zero including energy parameters plus with other related parameters for clauses (NC1, NC2, and NC3) together.

- i) NC1: Clauses with one literal which contributes to first-order connection strengths, (J_i)
- ii) NC2: Clauses with one literal which contributes to first-order connection strengths, (J_i) and second order strengths, (J_{ij})
- iii) NC3: Clauses with one literal which contributes to first-order connection strengths, (J_i) second order strengths, (J_{ij}) and third order strengths. (J_{ijk})

Next, let the network evolves or relaxes until minimum energy is reached. During the energy relaxation phase, the neuron state is updated by using Bipolar sigmoid activation function. After the network relaxed to an equilibrium state, test the final state obtained for the relaxed neuron whether it is a stable state. If the states remain unchanged for five time steps, then consider it as stable state. Subsequently, compute the corresponding final energy for the stable state. If the difference between the final energy and the global minimum energy is within the tolerance value, then consider the solution as global solution. Otherwise, we need to return back to beginning procedure. Then, we compute the ratio of global solutions and the hamming distance between stable state and global solution. Global solution is the interpretation used in the model for the given logic program or better known as global minima ratio. The formula is as follows:

$$\text{Global Minima Ratio} = \frac{\text{Number of global solutions}}{\text{Number of trials or iterations}} \quad (8)$$

Additionally, we run the relaxation for 1000 trials and 100 combinations to minimize the statistical error. The selected termination criterion is 0.001. Moreover, all these values are obtained by trial and error technique, where we tried several values as tolerance values and selected the value which yields better performance than other values. Finally, we compared the experimental results such as global minima ratio, Hamming distance and computation time between the McCulloch-Pitts function and Bipolar sigmoid activation function.

RESULTS AND DISCUSSION

We used the Microsoft Visual C++ 2013 as a platform of simulating, training and testing the network. The main objective of this computer simulation is to obtain the important outputs, such as global minima ratio, Hamming distance and computation time for both methods.

Global Minima Ratio

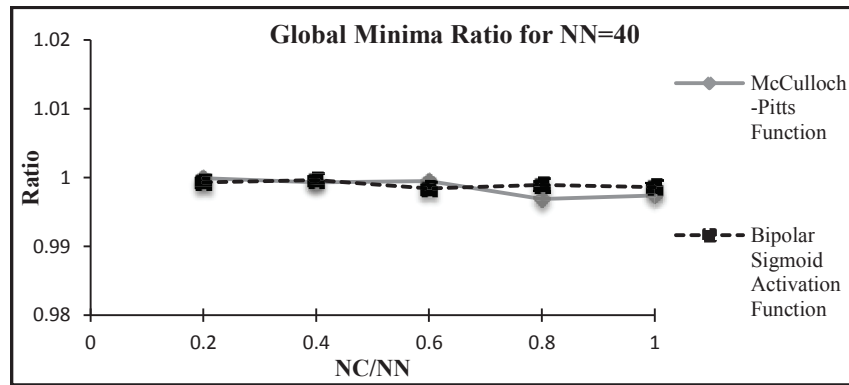


FIGURE 3. Graph of global minima ratio for NC1 given NN=40

According to Figure 3, it is obvious that Bipolar sigmoid activation function displays the best performance in global minima ratio aspect as well as the memory capacity compared to the McCulloch-Pitts functions. By using the Bipolar sigmoid activation function for neurons states are being updated more systematically. This is due to proper and steady neuron states guarantee the neuron to relax to the stable states after the energy relaxation looping. As the neurons already in steady state, due to effect of Bipolar sigmoid updating, so the neuron easily moved to the stable state during relaxation. For instance, the effective updating ability of the proposed model is a key to perform well in logic programming. Thus, the differences in the global solutions obtained are not much significant. According to Figure 3, we performed the simulation for NC1 (number of clauses equals to 1). In our study, we performed the simulation until NC3 (number of clauses=3) and number of neurons (NN) = 50. To sum up, Bipolar sigmoid activation function outperformed the McCulloch-Pitts function significantly.

Hamming Distance

Hamming distance can be defined as the closeness of a bit pattern to another bits pattern [16]. At this stage, hamming distance is measured between the final state and global state of the neurons upon relaxation [15].

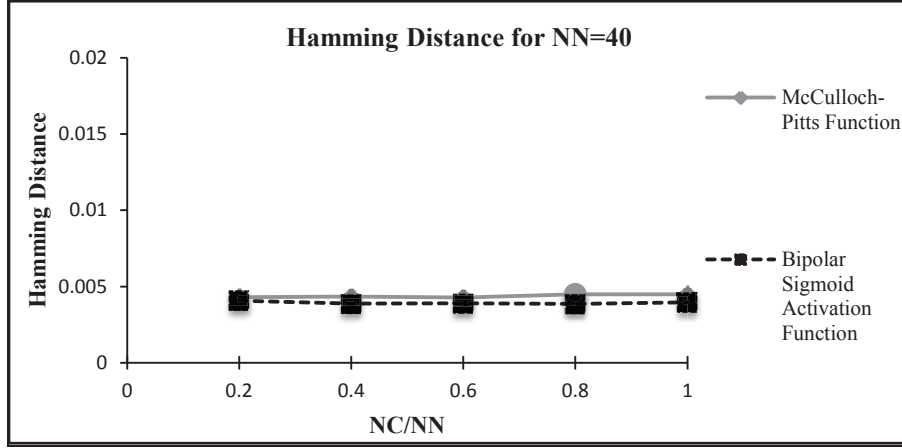


FIGURE 4. Hamming distance for NC1 given NN=40

Figure 4 illustrate the graph for the hamming distance obtained for Bipolar sigmoid activation function and McCulloch-Pitts function from the computer simulation that we have conducted. Hence, to avoid graphs overlapping, we only presented the result obtained for the number of neurons (NN) = 40. It shows that the value of hamming distance is approximately to zero for all the cases, (NC1 (one literals), NC2 (two literals), NC3 (three literals)). The underlying reason is due to all the neurons reached a stable state. However, by zooming Figure 4, it can be observed the proposed method (Bipolar sigmoid activation function) performs better since the hamming distances are much closer to zero compared to McCulloch-Pitts function, although we increased the network complexity by increasing the number of neurons (NN) and number of literals per clause (NC1, NC2, NC3). Therefore, Bipolar sigmoid activation function outperformed the McCulloch-Pitts function when doing logic programming in Hopfield network.

Computation Time Results

TABLE (1). The computational time (s) for Bipolar sigmoid activation function and McCulloch-Pitts function

Number of Neuron (NN)	Computational Time (s) for McCulloch-Pitts Function	Computational Time (s) for Bipolar Sigmoid Function
10	15	5
20	45785	12833
30	163706	93505
40	303458	204783
50	-	453917

The computation time or CPU time can be defined as the time taken for a logic program to generate the global solutions [16]. According to Table 1, the computation time for the Bipolar sigmoid activation function undergo significant boost as the number of neuron increases. However, we observed that for McCulloch-Pitts function, there is no computation time recorded when the number of neurons is equal to 50 ($NN = 50$). This is due to the condition when the network getting larger and complex, the network is likely to get stuck in local minima and devour more computation time. As a consequence of these arguments, extra time is needed to relax to global solution as the number of neurons increased. The neurons need to jump enormous energy barrier to reach the global solutions.

On the contrary, this will not happen in the cases of Bipolar Sigmoid Activation Function. This is because of Bipolar Sigmoid activation function can be used to update the synaptic strength in an optimized ways. The Bipolar sigmoid activation function has a powerful ability to reduce the computation burden for the training process [13]. It is clearly seen from Table 1 that the proposed model has faster computation time of doing logic programming in Hopfield network and able to obtain global minima solutions when the number of neurons (NN) equal to 50.

CONCLUSION

The computer simulations that had been carried out verified the powerful performance of Bipolar sigmoid activation function over the McCulloch-Pitts function. The global minima ratio, Hamming distance and computation time for the proposed model indicated significant difference in the performance compared with the McCulloch-Pitts function. According to the experimental results, Bipolar sigmoid activation function outperformed the McCulloch-Pitts function. All in all, we have authenticated that the Bipolar sigmoid activation function is a robust paradigm of doing logic programming in a Hopfield network.

ACKNOWLEDGMENTS

This research is partly financed by FRGS grant (203/ PMATHS/6711368) from the Ministry of Higher Education.

REFERENCES

1. R. Rojas, *Neural Networks: A Systematic Introduction*, (Springer, Berlin, 1996).
2. L. Altenberg, *Handbook of Evolutionary Computation*, (Oxford University Press, United Kingdom, 1997).
3. J. W. Lloyd, *Foundations of Logic Programming*, (Springer, Berlin, 1984).
4. W. A. T. Wan Abdullah, *International Journal of Intelligent System* **7**, 513-519 (1992).
5. J. J. Hopfield, *Procedings National Academy of Science USA* **79**, 2554-2558 (1982).
6. S. Haykin, *Neural Networks: A Comprehensive Foundation*, (Macmillan College Publishing, New York, 1999).
7. G. Pinkas, *Neural Computation* **3**, 282-291 (1991).
8. W. A. T. Wan Abdullah, *Physics Letters* **176**, 202-206 (1993).
9. S. Sathasivam, M. Velavan and P. F. Ng, *Modern Applied Science* **8**(13), 601-612 (2014).
10. R. A. Kowalski, *Logic Programming for Problem Solving*, (Elsevier Science Publishing, New York, 1979).
11. Kalman and S. C. Kwansy, *Proc of the International Joint Conference on Neural Network* **7**, 578-581 (1992).
12. W. A. Little, *Math. Biosci.* **19**, 101-120 (1974).
13. K. Bekir and A. O. Wehbi, *International Journal of Artificial Intelligence and Expert Systems* **1**(4), 111-122 (2010).
14. S. Zeng and T. Martinez, *Proc. of the International Conference on Neural Network and Genetic Algorithm*, 73-77 (1999).
15. S. Sathasivam and P. F. Ng, *Sains Malaysiana* **39**(1), 115-118 (2013).
16. S. Agantovic and R. Bresford, *Journal of Pharmaceutical and Biomedical Analysis* **1**, 717-727 (2000).